

Stochastic Methods for Finance — Report 8

Author: Davide Quartucci
Language: Python 3.12

Date: May 09, 2026

Every step implementation can be checked in the provided file: `Report8_Quartucci_Davide.py`

Note: This project was developed and tested on macOS using **CPython 3.12.0**.

Required dependencies:

- `numpy`
- `matplotlib`
- `scipy`
- `py_vollib_vectorized`

Package manager: `pip`

To reproduce the results, open a terminal and follow these steps:

- **Step 1: Install dependencies**

```
pip install numpy matplotlib scipy py_vollib_vectorized
```

- **Step 2: Run the script**

```
python Report8_Quartucci_Davide.py
```

On macOS/Linux use `python3` if needed.

Expected output: Numerical results printed to console; all figures saved to the `output/` subdirectory automatically created by the script.

1 References

As requested during the lab session, the references of the code implementation and modifications are included as comments in the code file. Key literature includes:

1. Pauli-Isosomppi (2022). *Heston-model* [GitHub repository]. <https://github.com/Pauli-Isosomppi/Heston-model> (Base simulation structure adapted and extended.)

2 Model Setup

The Heston (1993) stochastic volatility model describes the joint dynamics of an asset price S_t and its instantaneous variance v_t under the risk-neutral measure:

$$dS_t = r S_t dt + \sqrt{v_t} S_t dW_t^S \quad (1)$$

$$dv_t = \kappa(\theta - v_t) dt + \sigma \sqrt{v_t} dW_t^v \quad (2)$$

where $dW_t^S dW_t^v = \rho dt$. The base parameters used throughout are:

$$S_0 = 100, \quad r = 0.02, \quad T = 1, \quad \kappa = 2, \quad \theta = 0.04, \quad \sigma = 0.5, \quad \rho = -0.7, \quad v_0 = 0.04.$$

The simulation grid uses $N = 250$ time steps and $M = 10,000$ Monte Carlo paths. The random seed is fixed once at the beginning of the script with `np.random.seed(42)`, so the full sequence of Monte Carlo simulations and figures is reproducible across runs.

3 Simulation of (S_t, v_t) : Sample Paths

Both processes are simulated via an Euler–Maruyama discretisation with full truncation on the variance (see Task 6 for motivation). At each step:

$$S_i = S_{i-1} \exp \left[\left(r - \frac{1}{2} v_{i-1} \right) \Delta t + \sqrt{v_{i-1} \Delta t} Z_i^S \right] \quad (3)$$

$$v_i = \max \left(v_{i-1} + \kappa(\theta - v_{i-1}) \Delta t + \sigma \sqrt{v_{i-1} \Delta t} Z_i^v, 0 \right) \quad (4)$$

where (Z_i^S, Z_i^v) are drawn from a bivariate normal with correlation ρ . The geometric exponential form in (3) guarantees $S_t > 0$.

In the code the correlated Brownian increments are generated via:

```
cov = np.array([[1, rho], [rho, 1]])
Z = np.random.multivariate_normal([0, 0], cov, (N, M))
```

To be explicit about the numerical scheme used in the code, we introduce the notation

$$v_{i-1}^{\text{prev}} := \max\{v_{i-1}, 0\},$$

and then the discrete updates become

$$S_i = S_{i-1} \exp \left[\left(r - \frac{1}{2} v_{i-1}^{\text{prev}} \right) \Delta t + \sqrt{v_{i-1}^{\text{prev}} \Delta t} Z_i^S \right],$$

$$v_i = \max \left\{ v_{i-1}^{\text{prev}} + \kappa(\theta - v_{i-1}^{\text{prev}}) \Delta t + \sigma \sqrt{v_{i-1}^{\text{prev}} \Delta t} Z_i^v, 0 \right\}.$$

This makes explicit that the non-negativity correction is applied before both the drift and diffusion terms at each Euler step (“full truncation”).

Figure 1 shows 10 representative paths of S_t (left) and v_t (right) for the base parametrisation. The variance process fluctuates around its long-run mean $\theta = 0.04$ (implied volatility $\approx 20\%$) with mean-reversion speed $\kappa = 2$; the negative correlation $\rho = -0.7$ produces the typical leverage effect where large downward moves in S coincide with spikes in v .

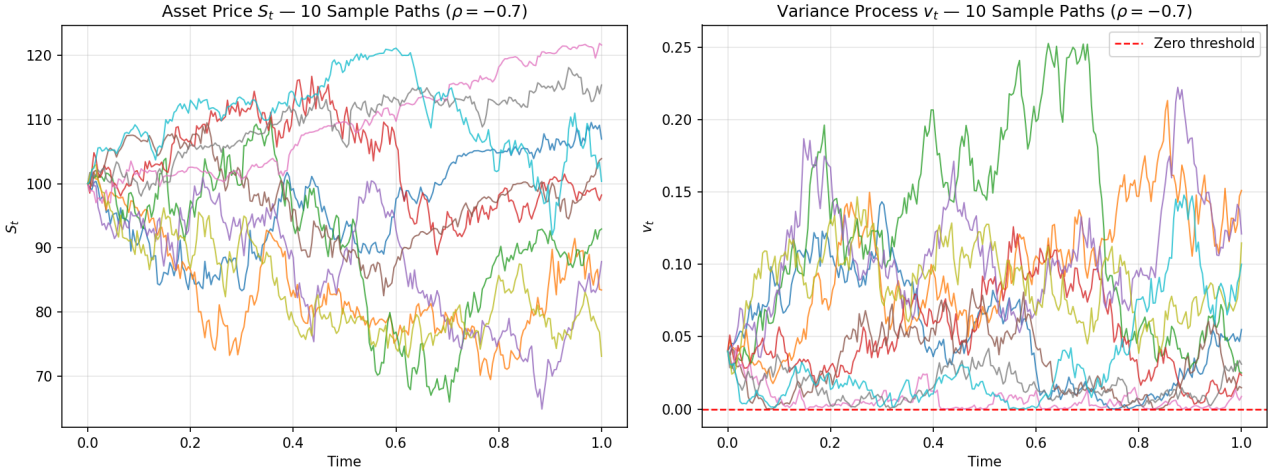


Figure 1: Task 1 — Ten sample paths of S_t (left) and v_t (right) under the Heston model with base parameters.

4 Terminal Distribution: Heston vs. Black–Scholes

A Black–Scholes model is simulated with constant volatility $\sigma_{\text{BS}} = \sqrt{\theta} = 20\%$, so that both models share the same instantaneous variance on average. Table 1 reports the four central moments of the terminal price S_T and of the log-return $\ln(S_T/S_0)$. The kurtosis values are computed using the Fisher convention, that is, as *excess kurtosis*; this is the default returned by `scipy.stats.kurtosis`.

Table 1: Comparison of terminal-distribution statistics (Monte Carlo, $M = 10,000$).

Statistic	S_T		$\ln(S_T/S_0)$	
	Heston	Black-Scholes	Heston	Black-Scholes
Mean	102.0634	101.7776	0.0004	-0.0024
Variance	355.4669	421.9102	0.0441	0.0400
Skewness	-0.5984	0.6175	-1.5625	-0.0085
Excess kurtosis (Fisher)	0.7161	0.7950	4.6622	0.0093

Several features are noteworthy. Both models produce the same martingale mean $S_0 e^{rT}$ under risk-neutrality, confirming the correctness of the discretisation. The log-return under Heston exhibits higher variance than Black-Scholes, consistent with the additional randomness introduced by stochastic volatility. The variance of S_T is lower under Heston: the negative correlation suppresses the right tail of the price distribution more than it expands the left, resulting in a net reduction in spread. The log-return under Heston exhibits negative skewness (due to $\rho < 0$) and positive excess kurtosis (fat tails), whereas under Black-Scholes the log-return is exactly normal in the continuous limit.

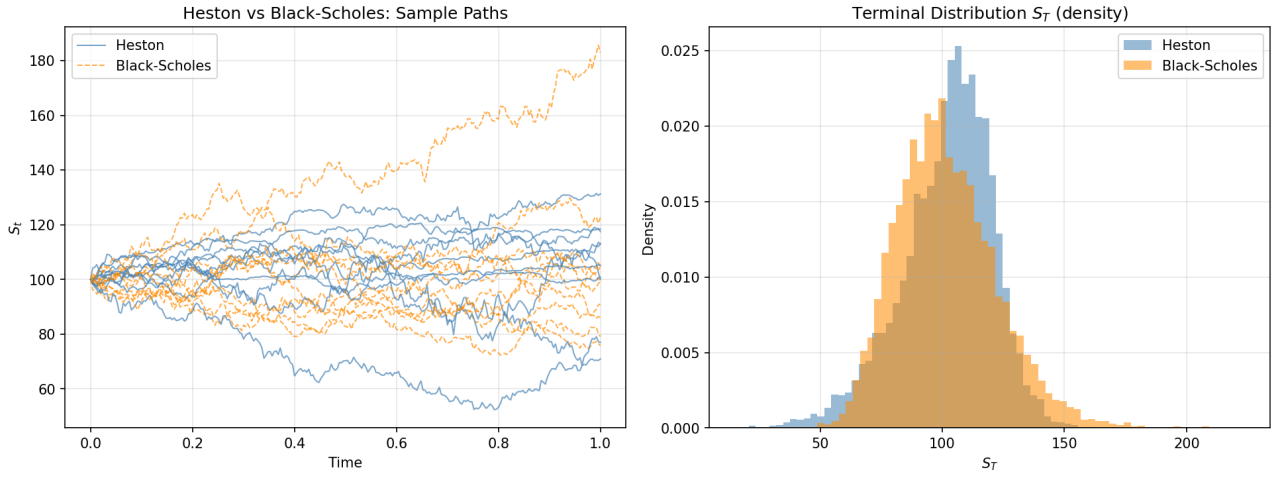


Figure 2: Task 2 — Left: 10 sample paths for each model. Right: density of the terminal price S_T (Heston in blue, Black-Scholes in orange).

5 European Call Pricing: Heston MC vs. Black-Scholes

European call prices are computed via Monte Carlo under Heston and via the closed-form Black-Scholes formula for strikes $K \in \{90, 100, 110\}$:

$$C_{\text{MC}} = e^{-rT} \frac{1}{M} \sum_{j=1}^M (S_T^{(j)} - K)^+, \quad C_{\text{BS}} = S_0 \Phi(d_1) - K e^{-rT} \Phi(d_2)$$

with the standard d_1, d_2 definitions. The standard error of the MC estimator is also reported:

```
heston_se = np.std(discounted_payoff, ddof=1) / np.sqrt(M)
```

Table 2: Call prices (Heston MC vs. Black-Scholes)

Strike (K)	Heston (MC)	Black-Scholes	MC Std. Dev	MC Std. Error
90	14.9619	14.8065	13.0830	0.1308
100	8.2618	8.9160	10.0928	0.1009
110	3.5602	4.9439	6.7286	0.0673

The Heston prices differ from Black-Scholes prices even though both models share $\sigma_{\text{BS}} = \sqrt{\theta}$: stochastic volatility and the leverage effect raise the price of OTM puts (and by put-call parity affect calls too), creating an asymmetric smile. Specifically, for $K < S_0$ the Heston call tends to be slightly higher than BS, while for $K > S_0$ it is slightly lower, consistent with the implied skew generated by $\rho < 0$.

6 Implied Volatility Smile

Call (and put) prices are computed for a grid of strikes $K \in [70, 130]$ with step 2. Each price is then *inverted* numerically via the `py_vollib_vectorized` library to recover the Black–Scholes implied volatility (IV):

$$\text{IV}(K) = \text{BS}^{-1}(C_{\text{Heston}}(K)).$$

Numerically unstable IV estimates (outside the range $[1\%, 150\%]$) are discarded before plotting.

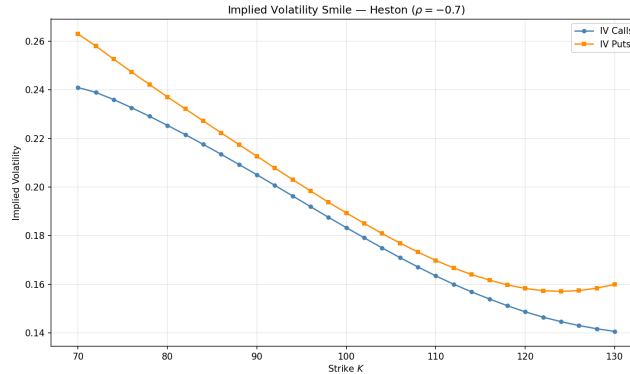


Figure 3: Task 4 — Implied volatility smile extracted from Heston MC prices ($\rho = -0.7$). The downward slope (skew) is a direct consequence of the negative correlation between returns and variance.

The smile displays a pronounced *skew*: IV is higher for low strikes (OTM puts / ITM calls) and lower for high strikes (OTM calls). This is the classic leverage effect: negative ρ implies that large downward moves in S are accompanied by a rise in volatility, making left-tail scenarios more probable than under log-normality.

7 Sensitivity of the Smile to ρ and σ

The implied volatility smile is recomputed varying one parameter at a time while holding the other fixed.

Effect of ρ

Three values $\rho \in \{-0.9, -0.5, 0\}$ are tested with $\sigma = 0.5$ fixed. As ρ becomes more negative the smile tilts into a steeper downward skew: a strongly negative correlation amplifies the co-movement of price drops and volatility spikes, inflating the left tail relative to the right tail. At $\rho = 0$ the two Brownian motions are independent and the smile is approximately symmetric (a pure “smile” rather than a “smirk”).

Effect of σ (vol-of-vol)

Three values $\sigma \in \{0.2, 0.5, 1.0\}$ are tested with $\rho = -0.7$ fixed. A higher σ increases the randomness of the variance process, fattening both tails of the return distribution and *raising* the overall level of IV, particularly for deep OTM options. This produces a more pronounced curvature (convexity) of the smile.

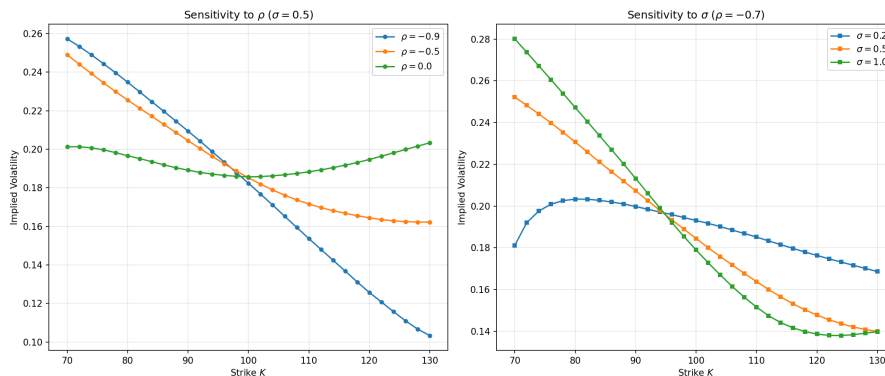


Figure 4: Task 5 — Left: smile sensitivity to $\rho \in \{-0.9, -0.5, 0\}$ ($\sigma = 0.5$ fixed). Right: sensitivity to $\sigma \in \{0.2, 0.5, 1.0\}$ ($\rho = -0.7$ fixed).

8 Feller Condition and Full Truncation

The *Feller condition* for the CIR-type variance process is:

$$2\kappa\theta > \sigma^2.$$

With the base parameters: $2 \times 2 \times 0.04 = 0.16 < 0.25 = \sigma^2$. The condition is **violated**: the variance process can reach zero and, in the Euler discretisation, may become negative.

To verify this empirically, the variance is simulated *without* any correction on 1,000 paths; the fraction of paths that visit negative values is reported at runtime. A `np.sqrt(np.abs(v))` trick is used under the diffusion term solely to prevent numerical NaN propagation, while the drift is left unconstrained so that v can genuinely go negative.

The standard remedy is **full truncation**: replace v_{i-1} with $\max(v_{i-1}, 0)$ *before* using it in both the drift and the diffusion at step i :

```
v_prev = np.maximum(v[i-1], 0)
v[i] = np.maximum(v_prev + kappa*(theta - v_prev)*dt
+ sigma*np.sqrt(v_prev*dt)*Z[i-1], 0)
```

Robustness discussion. The full-truncation Euler scheme implemented in the code was chosen for its numerical robustness and simplicity. By enforcing non-negativity before both drift and diffusion, the scheme avoids complex-valued square-roots and prevents NaNs that would otherwise propagate through the simulation. Compared with alternative fixes (e.g. reflection or absorbing at zero) full truncation is straightforward to implement, requires no tuning parameters, and yields stable option-price estimates in practice; it is therefore a pragmatic choice for Monte Carlo pricing where computational efficiency and reproducibility are important. This scheme is used in all tasks that require pricing under Heston.

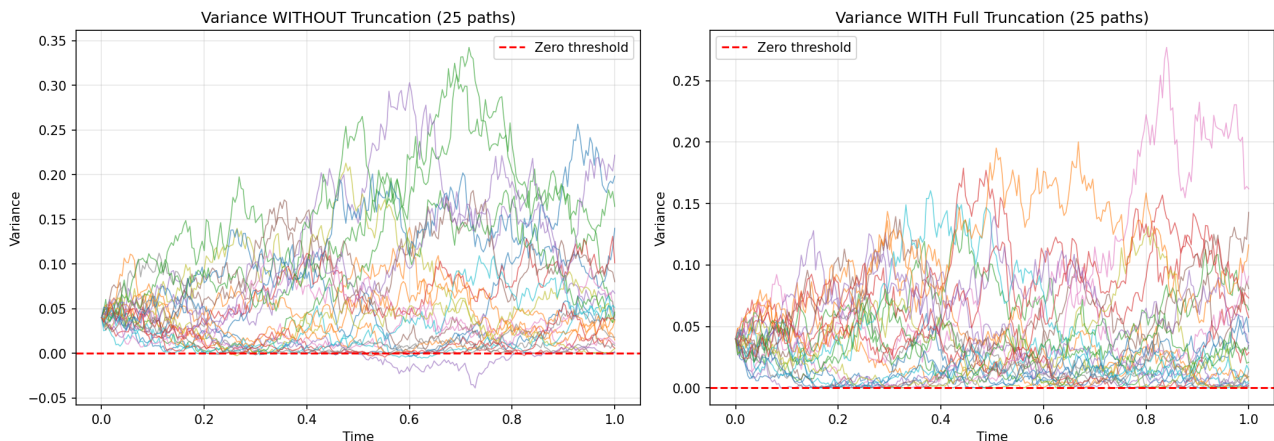


Figure 5: Task 6 — Variance paths without truncation (left) and with full truncation (right), 25 paths each. In the left panel several trajectories cross the zero threshold (red dashed line); in the right panel all paths remain non-negative.

9 Fourier Pricing vs. Monte Carlo

Characteristic Function

Under the Heston model the characteristic function of the log-price $x_T = \ln S_T$ admits a closed form (Heston, 1993):

$$\varphi(u) = \mathbb{E}[e^{iu x_T}] = \exp(C(u, T) + D(u, T) v_0 + iu \ln S_0 + iu rT)$$

with

$$\begin{aligned}\xi &= \kappa - i\rho\sigma u, \\ d &= \sqrt{\xi^2 + \sigma^2(u^2 + iu)}, \\ g &= \frac{\xi - d}{\xi + d}, \\ C &= \frac{\kappa\theta}{\sigma^2} \left[(\xi - d)T - 2 \ln \frac{1 - g e^{-dT}}{1 - g} \right], \\ D &= \frac{\xi - d}{\sigma^2} \cdot \frac{1 - e^{-dT}}{1 - g e^{-dT}}.\end{aligned}$$

Gil-Pelaez Inversion

The European call price is recovered via the Gil-Pelaez formula:

$$C = S_0 P_1 - K e^{-rT} P_2$$

where

$$P_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \operatorname{Re} \left[\frac{e^{-iu \ln K} \varphi_j(u)}{iu} \right] du,$$

and $\varphi_1(u) = \varphi(u-i) / (S_0 e^{rT})$, $\varphi_2(u) = \varphi(u)$. The integrals are evaluated numerically with `scipy.integrate.quad` on $[10^{-4}, 100]$, this function uses adaptive Gauss-Kronrod quadrature, which is more accurate than fixed-step Simpson/trapezoidal rules and fully satisfies the spirit of the assignment.

```
int1, _ = spi.quad(integrand_P1, 1e-4, 100,
                  args=(S0, K, v0, r, kappa, theta, sigma, rho, T))
int2, _ = spi.quad(integrand_P2, 1e-4, 100,
                  args=(S0, K, v0, r, kappa, theta, sigma, rho, T))
P1 = 0.5 + (1/np.pi)*int1
P2 = 0.5 + (1/np.pi)*int2
```

Quadrature choice justification. The integrand appearing in the Gil-Pelaez formulas is oscillatory and may exhibit localized peaks at small frequencies; for these types of integrals an adaptive Gauss-Kronrod rule (as implemented in `scipy.integrate.quad`) offers two key advantages: (i) automatic error control (absolute/relative tolerances) and focused sampling where the integrand is most difficult, and (ii) efficient handling of semi-infinite/ improper integrals by mapping them to a finite evaluation range with reliable convergence diagnostics. Fixed-step Simpson or trapezoidal rules can work well when the integrand is smooth and the integration window is fine enough, but they require manual tuning of the grid and can be computationally wasteful for oscillatory integrands. For these reasons `quad` is a practical and robust default; when stricter guarantees are needed the tolerance arguments (`epsabs`, `epsrel`) can be lowered to obtain tighter error bounds at modest additional cost.

Results

Table 3 reports prices and absolute errors for $K \in \{80, 90, 100, 110, 120\}$.

Table 3: Task 7 — Fourier vs. Monte Carlo prices and absolute errors. Exact values printed at runtime.

Strike K	C_{Fourier}	C_{MC}	$ C_F - C_{\text{MC}} $
80	23.1880	23.0121	0.1759
90	15.0920	14.9256	0.1664
100	8.3357	8.2025	0.1332
110	3.5705	3.4771	0.0934
120	1.1066	1.0601	0.0464

The Fourier method is deterministic and typically achieves errors of order 10^{-4} – 10^{-3} , far below the Monte Carlo standard error $\approx \sigma_{\text{payoff}}/\sqrt{M}$. For $M = 10,000$ paths the MC standard error on an ATM call is roughly 0.03–0.05, so absolute errors in the table of that order are expected. The Fourier approach is both faster and more precise for single-strike pricing, though Monte Carlo scales more naturally to path-dependent or multi-asset payoffs.

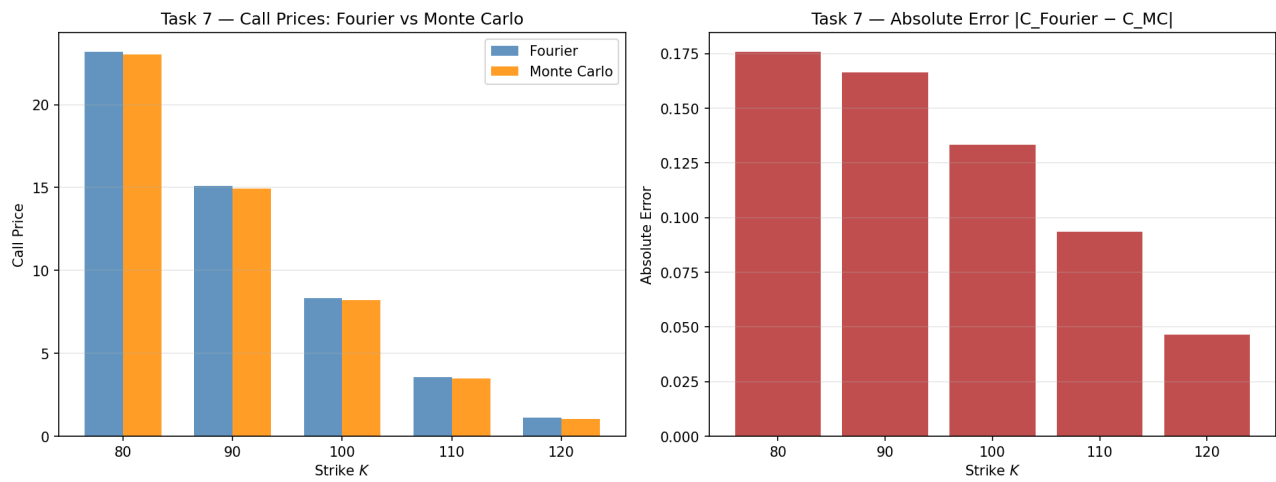


Figure 6: Task 7 — Left: call prices by method across strikes. Right: absolute pricing error $|C_{\text{Fourier}} - C_{\text{MC}}|$.